

# BCA152 Laboratory Activity 1

## FreeRTOS Multisensor Room Monitor — Laboratory Report

**Submitted by:** Sofiah Mariell Silao

**Submitted to:** Asst. Prof. Paul Rodolf P. Castor, M.Sc.

**Subject Code/Section:** BCA152 - B186.1

**Platform:** ESP32 DevKit, ESP-IDF, FreeRTOS, PlatformIO, and Wokwi

**Date:** 24 September 2026

### 1. Problem and Requirements

The aim of this laboratory activity was to build a room-monitoring system and practice organizing its work as FreeRTOS tasks. The system reads temperature and humidity from a DHT22, reads a photoresistor through the ESP32 ADC, displays information on an OLED, accepts page changes from a rotary encoder, detects motion with a PIR sensor, and sounds a buzzer when temperature is outside the normal range.

The system must share readings safely between tasks and use motion to switch between ACTIVE and INACTIVE states. After 15 seconds without motion, the OLED is cleared. When the PIR detects motion again, the system returns to ACTIVE. A temperature below 18 °C or above 30 °C activates the alarm. Readings at either limit are considered normal.

Wokwi was used instead of a physical ESP32 circuit. Its controls were used to change the simulated temperature and humidity and to trigger the PIR and encoder. The LDR level is shown as a normalized ADC percentage. It is not calibrated to lux.

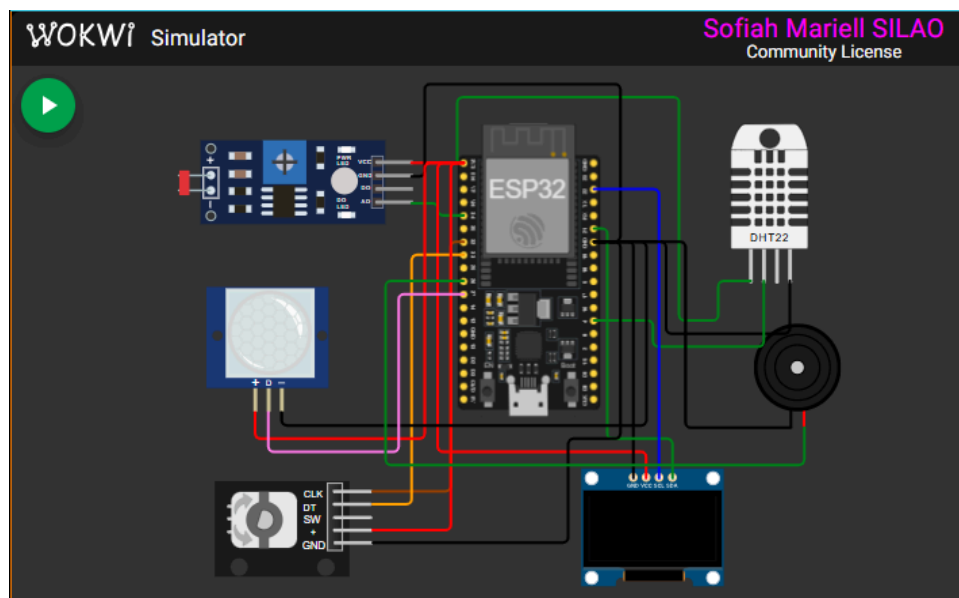


Figure 1. Wokwi circuit showing the ESP32, sensors, OLED, encoder, and buzzer.

## 2. System Architecture and Design

### Hardware design

| Device or signal  | ESP32 pin or interface           | Purpose                                         |
|-------------------|----------------------------------|-------------------------------------------------|
| DHT22 data        | GPIO 4                           | Temperature and relative humidity               |
| LDR analog output | GPIO 34, ADC1 channel 6          | Relative light level, displayed as a percentage |
| SSD1306 OLED      | I2C SDA 21, SCL 22, address 0x3C | Displays the selected reading or motion status  |
| KY-040 encoder    | CLK 32, DT 33                    | Selects the display page                        |
| PIR output        | GPIO 27                          | Detects motion                                  |
| Buzzer            | GPIO 26, LEDC PWM                | Temperature alarm output                        |

All components share ESP32 power and ground connections in the simulation.

### Software design

app\_main() initializes shared FreeRTOS objects, sensors, and the buzzer, then creates the worker tasks. The code is split into modules for sensors, display, input, alarm, motion, state decisions, and shared RTOS objects.

DisplayTask is the only task that writes to the OLED. Other tasks send it data or state through queues and the event group. Temperature and motion-state decisions are kept in separate functions so they can be tested without the ESP32.

### System state machine

| Current state | Condition                | Next state and action                           |
|---------------|--------------------------|-------------------------------------------------|
| INACTIVE      | PIR detects motion       | ACTIVE; set EVENT_ACTIVE and refresh the OLED   |
| ACTIVE        | Motion continues         | Stay ACTIVE and restart the inactivity timer    |
| ACTIVE        | No motion for 15 seconds | INACTIVE; clear EVENT_ACTIVE and clear the OLED |
| INACTIVE      | No motion                | Stay INACTIVE; continue monitoring the PIR      |

The firmware starts in **INACTIVE**. Sensor sampling and alarm checking continue in both states, while the OLED is blanked in **INACTIVE**.

### 3. FreeRTOS Architecture

A task is **RUNNING** while it uses a CPU core, **READY** when it can run but is waiting for a core, and **BLOCKED** while it waits for a delay or queue event.

| Task        | Responsibility                                                           | Trigger/period                        | Priority | Communication                                           | Typical blocked condition               |
|-------------|--------------------------------------------------------------------------|---------------------------------------|----------|---------------------------------------------------------|-----------------------------------------|
| SensorTask  | Reads DHT22 and LDR; publishes valid sensor data and updates alarm state | Every 2 seconds                       | 2        | Sensor queue and event group                            | vTaskDelayUntil() until the next sample |
| DisplayTask | Owens OLED and shows the selected page while <b>ACTIVE</b>               | Checks queue updates and system state | 1        | Reads sensor queue, display-mode queue, and event group | Waits up to 100 ms for sensor data      |
| InputTask   | Polls encoder and publishes the selected page                            | Short periodic polling                | 3        | Display-mode queue                                      | 1-tick vTaskDelay()                     |
| MotionTask  | Polls PIR and updates <b>ACTIVE/MOTION</b> state                         | Every 100 ms                          | 3        | Event group                                             | 100 ms vTaskDelay()                     |
| AlarmTask   | Checks alarm state and controls buzzer PWM                               | Checks every 100 ms                   | 2        | Reads event group                                       | 100 ms vTaskDelay()                     |

## Priority justification

MotionTask and InputTask use priority 3 because the PIR and encoder should be handled promptly. SensorTask has priority 2 because it samples every two seconds. AlarmTask also has priority 2 and checks the alarm event every 100 ms. DisplayTask has priority 1 because a screen update can tolerate more delay than input handling or sensor acquisition.

These priorities follow the laboratory's suggested starting values. They do not mean the display is unimportant. Priority affects scheduling when tasks are ready on the same core; blocking and the ESP32's dual-core scheduler also affect timing.

## Inter-task communication

- `sensor_data_queue` holds one latest `SensorData` sample. `SensorTask` overwrites it, and `DisplayTask` reads it.
- `display_mode_queue` holds the latest encoder selection. `InputTask` overwrites it, and `DisplayTask` reads it.
- `system_events` holds three bits. `MotionTask` sets and clears `EVENT_ACTIVE` and `EVENT_MOTION`. `SensorTask` sets and clears `EVENT_ALARM`. `DisplayTask` reads the active state, `SensorTask` copies the motion state into its data sample, and `AlarmTask` reads the alarm state.
- `serial_mutex` is used by the `SERIAL_LOGI`, `SERIAL_LOGW`, and `SERIAL_LOGE` macros to protect application log calls.

## Why `vTaskDelayUntil()` is used

`vTaskDelayUntil()` schedules the next wake relative to a saved tick time. This helps keep sensor sample starts on a regular schedule. A simple `vTaskDelay()` waits after the task finishes its work, so the time spent reading sensors is added to each cycle. Over many cycles, that can make sampling drift later. `vTaskDelayUntil()` does not prevent late cycles if work takes too long, but it avoids adding normal execution time to every period.

## 4. Implementation

`SensorTask` reads the DHT22 and ADC, combines valid readings into one `SensorData` sample, and sends it through the one-item queue. Overwriting the queue keeps the display current instead of building up old readings. If either sensor read fails, that sample is not published.

The LDR percentage is based on the ADC raw value. It shows relative change in the simulation but is not a calibrated light measurement. The encoder uses the `DisplayMode` enumeration to represent Temperature, Humidity, Light, and Motion pages.

Temperature decisions return NORMAL, LOW\_TEMPERATURE, or HIGH\_TEMPERATURE. SensorTask sets or clears the alarm event, and AlarmTask controls the buzzer with a 2 kHz, 50% duty PWM signal. This keeps the temperature decision separate from the buzzer hardware.

### Mutex explanation

The shared resource is the serial diagnostic output. SensorTask, InputTask, MotionTask, AlarmTask, DisplayTask, and startup code can all write log messages. The mutex in the serial logging macros makes protected calls take turns using that output.

During the mutex-removal experiment, the captured log lines remained readable and did not visibly interleave. ESP-IDF may serialize individual logging calls internally. The explicit mutex remains in the final design to demonstrate and document coordination of this shared resource.

## 5. Verification and Testing

### Unit tests

The native PlatformIO environment ran 13 Unity tests successfully:

- Five temperature tests: below the lower limit, exactly at the lower limit, inside the normal range, exactly at the upper limit, and above the upper limit.
- Four navigation tests: forward and reverse page changes, including both wrap directions.
- Four state tests: remaining ACTIVE before timeout, becoming INACTIVE at timeout, remaining INACTIVE without motion, and returning to ACTIVE when motion is detected.

These tests use decision logic that can run on the computer without an ESP32 or Wokwi.

### Wokwi functional tests

| ID    | Stimulus                 | Observed result                                                       | Result/evidence  |
|-------|--------------------------|-----------------------------------------------------------------------|------------------|
| FT-01 | Change temperature       | OLED temperature page updated to the selected simulated value.        | PASS — FT-01.png |
| FT-02 | Change humidity          | Humidity reading changed; values in the run included 61.2% and 92.0%. | PASS — FT-02.png |
| FT-03 | Change light input       | Light page changed; one run showed 16% ADC level.                     | PASS — FT-03.png |
| FT-04 | Rotate encoder clockwise | Next display page was selected.                                       | PASS — FT-04.png |

|       |                                 |                                                                                         |                   |
|-------|---------------------------------|-----------------------------------------------------------------------------------------|-------------------|
| FT-05 | Rotate encoder counterclockwise | Previous display page was selected.                                                     | PPASS — FT-05.png |
| FT-06 | Set temperature above 30 °C     | At 31.0 °C, the alarm activated and the buzzer was driven.                              | PASS — FT-06.png  |
| FT-07 | Return temperature to normal    | At 23.5 °C, the alarm cleared and the buzzer stopped.                                   | PASS — FT-07.png  |
| FT-08 | Trigger PIR                     | Motion was logged, the system became ACTIVE, and the OLED showed motion status.         | PASS — FT-08.png  |
| FT-09 | Wait for inactivity timeout     | After about 15 seconds without motion, the OLED went blank.                             | PASS — FT-09.png  |
| FT-10 | Trigger PIR while INACTIVE      | Motion returned the system to ACTIVE and the OLED resumed displaying the selected page. | PASS — FT-10.png  |

The observed values depend on the Wokwi settings. The test screenshots are stored in docs/images.

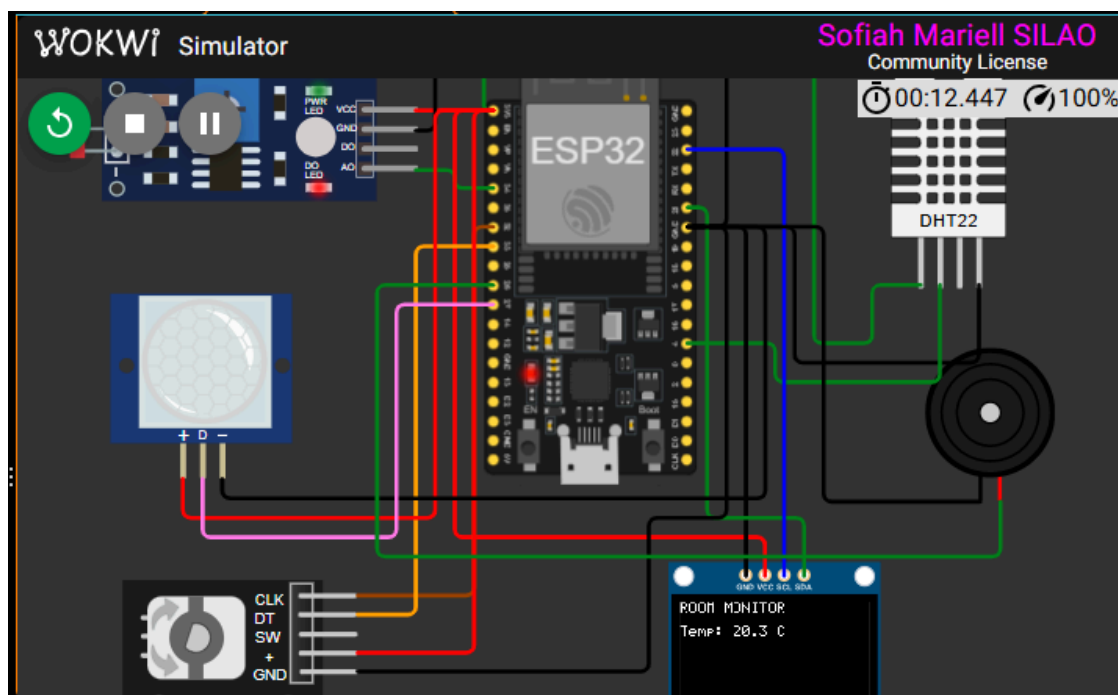


Figure 2. Running simulation with a temperature reading shown on the OLED.

## Deliberate FreeRTOS fault experiments

| Experiment         | Temporary change and observation                                                                                                          | Meaning and final state                                                                                                                                                                                              |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Remove blocking | The 1-tick delay was removed from InputTask. The OLED became blank and the serial monitor showed a backtrace while sensor logs continued. | Without a blocking point, InputTask stayed runnable and used more CPU. Lower-priority work on the same core could wait longer. The screenshot does not identify the backtrace's root cause. The delay was restored.  |
| 2. Change priority | InputTask priority was temporarily raised from 3 to 5. A delay was observed, but the affected output was not identified in the notes.     | A higher-priority ready task can reduce CPU time for lower-priority tasks on the same core. The ESP32 has two cores, and InputTask still delayed each loop, so the visible effect can vary. Priority 3 was restored. |
| 3. Remove mutex    | The serial macros logged without taking serial_mutex. The captured output did not show visibly interleaved lines.                         | Individual logging calls may be serialized by the logging implementation, so no visible difference is possible. The mutex was restored afterward.                                                                    |

The fault changes were temporary and were restored. The normal implementation uses `vTaskDelay(1)` in InputTask, priority 3, and mutex-protected serial logging.

Evidence screenshots: Fault experiment 1.a.png, Fault experiment 2.a.png, and Fault experiment 3.a.png in docs/images.

## 6. Static Code Analysis

PlatformIO's `pio check -e esp32dev` completed successfully after package scanning was skipped with `check_skip_packages = yes`. The remaining messages were low-severity Cppcheck style findings. No high- or medium-severity issues were reported in the final check.

| Finding                                             | File / line                        | Cause                                                      | Resolution / interpretation                                   |
|-----------------------------------------------------|------------------------------------|------------------------------------------------------------|---------------------------------------------------------------|
| 1. <code>evaluateTemperature</code> reported unused | <code>src/alarm_logic.cpp:9</code> | Cppcheck did not see its references in other source files. | Kept; SensorTask and native tests call it through its header. |

|                                        |                         |                                                 |                                                   |
|----------------------------------------|-------------------------|-------------------------------------------------|---------------------------------------------------|
| 2. nextDisplayMode reported unused     | src/system_state.cpp:3  | Cross-file use was not visible to this finding. | Kept; InputTask and navigation tests call it.     |
| 3. previousDisplayMode reported unused | src/system_state.cpp:16 | Cross-file use was not visible to this finding. | Kept; InputTask and navigation tests call it.     |
| 4. evaluateSystemState reported unused | src/system_state.cpp:29 | Cross-file use was not visible to this finding. | Kept; MotionTask and state tests call it.         |
| 5. DisplayModeName reported unused     | src/system_state.cpp:47 | Cross-file use was not visible to this finding. | Kept; InputTask calls it for encoder diagnostics. |

These functions are used by other modules. Removing them just to silence the style messages would break working code or tests, so the findings were reviewed rather than treated as runtime defects.

## 7. Engineering Discussion

**Debugging:** Early DHT22 reads timed out while waiting for a sensor response. Checking the Wokwi wiring and matching the data connection to GPIO 4 resolved the reading failure. The SSD1306 component manager also had an extraction problem during setup; after the dependency state was corrected, the ESP32 build succeeded. These problems showed that a successful compile alone does not confirm correct wiring or runtime behavior.

**Queue length:** One-item queues were used because the display needs the latest sensor sample and page selection. A longer queue could cause the display to process stale values after quick changes.

**Display ownership:** Only DisplayTask writes to the OLED. If several tasks wrote to it directly, they would need extra synchronization and could overwrite each other's display contents.

**Limitations:** Testing was done in Wokwi, not on physical hardware. The LDR percentage is not calibrated to lux, and the buzzer sound level and PIR range were not measured on a real board. The cause of the backtrace in Experiment 1 was not determined. The delayed response in Experiment 2 was observed but not traced to a specific task.

**Possible improvements:** Test on physical hardware, calibrate the LDR, consider alarm hysteresis near the thresholds, measure task stack high-water marks and response times, and decode the Experiment 1 backtrace using the matching firmware ELF file.

## 8. Conclusion

This activity showed how to split a sensor application into FreeRTOS tasks and how queues, event groups, and a mutex can support communication between them. The native tests made temperature, navigation, and state decisions easier to check without hardware. Wokwi testing confirmed the recorded sensor display, alarm, encoder, and PIR behavior.

The fault experiments showed that blocking points and priorities can affect responsiveness, although the visible results depend on task readiness, the ESP32's two cores, and simulation behavior. No obvious log interleaving was seen when the mutex was removed. Further work should include physical hardware tests, measured timing, LDR calibration, and investigation of the delay-free run's backtrace.

## Appendix A — Requirements Traceability.

The requirement IDs below follow the mappings provided in the laboratory instructions. FR-09 is implemented by MotionTask calling the pure state-decision function; this project does not have a separate StateTask.

| Requirement                | Implementation                                              | Verification evidence                           |
|----------------------------|-------------------------------------------------------------|-------------------------------------------------|
| FR-01: sensor measurements | SensorTask, sensors.cpp, dht22.cpp                          | FT-01 to FT-03; five temperature decision tests |
| FR-05: display readings    | DisplayTask and sensor_data_queue                           | FT-01 to FT-03; OLED screenshots                |
| FR-06: encoder navigation  | InputTask, display_mode_queue, page-navigation functions    | Four navigation tests; FT-04 and FT-05          |
| FR-07: temperature alarm   | evaluateTemperature, EVENT_ALARM, AlarmTask, GPIO 26 buzzer | Five alarm tests; FT-06 and FT-07               |
| FR-09: motion/system state | MotionTask, evaluateSystemState, EVENT_ACTIVE               | Four state tests; FT-08 to FT-10                |